Submission Worksheet

Submission Data

Course: IT114-007-F2025
Assignment: IT114 Module 4 Sockets Part3 Challenge
Student: Evans M. (ekn3)
Status: Submitted | **Worksheet Progress:** 100%
Potential Grade: 10.00/10.00 (100.00%)
Received Grade: 0.00/10.00 (0.00%)
Started: 10/21/2025 9:03:25 PM
Updated: 10/22/2025 12:34:05 AM
Grading Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-module-4-sockets-part3-challenge/grading/ekn3
View Link: https://learn.ethereallab.app/assignment/v3/IT114-007-F2025/it114-module-4-sockets-part3-challenge/view/ekn3

Instructions

- Overview Link: https://youtu.be/_029E_aBTFo

1. Ensure you read all instructions and objectives before starting.
2. Create a new branch from `main` called `M4-Homework`
    1. `git checkout main` (ensure proper starting branch)
    2. `git pull origin main` (ensure history is up to date)
    3. `git checkout -b M4-Homework` (create and switch to branch)
3. Copy the template code from here: GitHub Repository - M4 Homework
    - It includes Sockets `Part1`, `Part2`, and `Part3`. Put all into an `M4` folder or similar if you don't have them yet (adjust `package` reference at the top if you chose a different folder name).
    - Make a copy of `Part3` and call it `Part3HW`
        - Fix the `package` and `import` references at the top of each file in this new folder (Note: you'll only be editing files in `Part3HW`)
    - Immediately record to history
        - `git add .`
        - `git commit -m "adding M4 HW baseline files"`
        - `git push origin M4-Homework`
        - Create a Pull Request from `M4-Homework` to `main` and keep it open
4. Fill out the below worksheet
    - Each Problem requires the following as you work
        - Ensure there's a comment with your UCID, date, and brief summary of how the problem was solved
        - Code solution (add/commit periodically as needed)
        - Hint: Note how `/reverse` is handled
5. Once finished, click "Submit and Export"
6. Locally add the generated PDF to a folder of your choosing inside your repository folder and move it to Github
    1. `git add .`
    2. `git commit -m "adding PDF"`

3. `git push origin M4-Homework`
4. On Github merge the pull request from `M4-Homework` to `main`

7. Upload the same PDF to Canvas
8. Sync Local
    1. `git checkout main`
    2. `git pull origin main`

Section #1: (3 pts.) Challenge 1 - Coin Flip

Progress: 100%

Task #1 (3 pts.) - Implement a Coin Flip Command

Progress: 100%

Details:

- `Client` must capture the user entry and generate a valid command per the lesson details
    - Command format must be `/flip`
- `ServerThread` must receive the data and call the correct method on `Server`
- `Server` must expose a method for the logic and send the result to everyone
    - The message must be in the format of `<who> flipped a coin and got <result>` and be from the Server
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:
Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from `Client`
    - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from `ServerThread`
    - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from `Server`
    - Should only need to create a new method and pass the result message to `relay()`
4. Show 5 examples of the command being seen across all terminals (2+ Clients and 1 Server)
    1. This can be captured in one screenshot if you split the terminals side by side



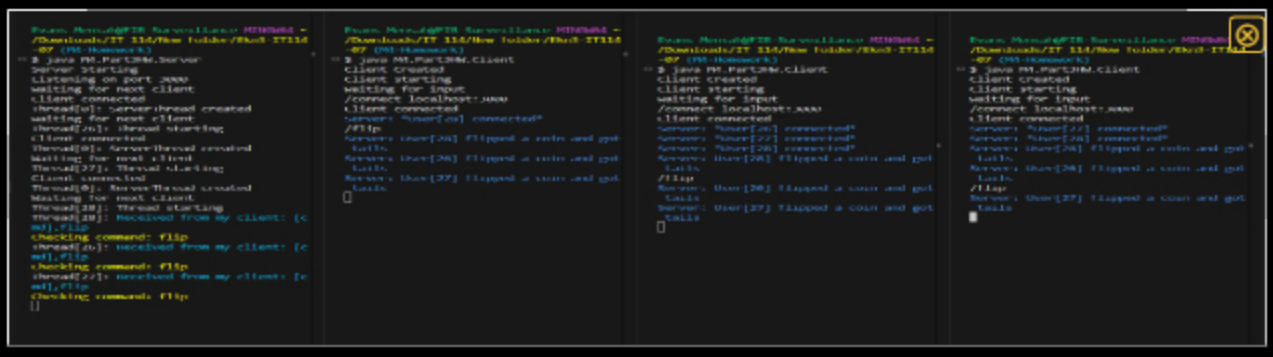
Solution code from Client



Solution code from ServerThread

```java
/**
 * Expose access to the disconnect action
 *
 * @param serverThread
 */
//ekn3 10-12-25
protected synchronized void handleFlip(ServerThread sender){
    String who = sender == null ? "Server" :String.format(format:"User[%s]",sender.getClientId());
    String result = Math.random() < 0.5 ? "heads" : "tails";
    String message = String.format(format:"%s flipped a coin and got %s",who,result);
    relay(sender:null, message);        You, 32 minutes ago • Uncommitted changes
} <- #120-125 protected synchronized void handleFlip(ServerThread sender)
protected synchronized void handleDisconnect(ServerThread sender) {
    disconnect(sender);
}
```

Solution code from Server



Solution output

Saved: 10/22/2025 12:16:19 AM

Part 2:

Progress: 100%

Details:
Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1
https://github.com/Faculty06/Ekn3-IT114-M4-Homework/M4/Part3HW/Client.java

URL
https://github.com/Faculty06/Ekn

Saved: 10/22/2025 12:16:19 AM

Part 3:

Progress: 100%

Details:
Briefly explain `how` the code solves the challenge (note: this isn't the same as `what` the code does)

Your Response:

The server thread detects the flip command checks it and run on the server and output a response of a random coin toss

Saved: 10/22/2025 12:16:19 AM

Section #2: (3 pts.) Challenge 2 - Private Message

Progress: 100%

Task #1 (3 pts.) - Implement a Private Message Command

Progress: 100%

Details:

- `Client` must capture the user entry and generate a valid command per the lesson details
  - Command format must be `/pm <target id> <message>`
- `ServerThread` must receive the data and call the correct method on `Server`
- `Server` must expose a method for the logic
  - The message must be in the format of `PM from <who>: <message>` and be from the Server
  - The result must only be sent to the original sender and to the receiver/target
- Add code to solve the problem (add/commit as needed)

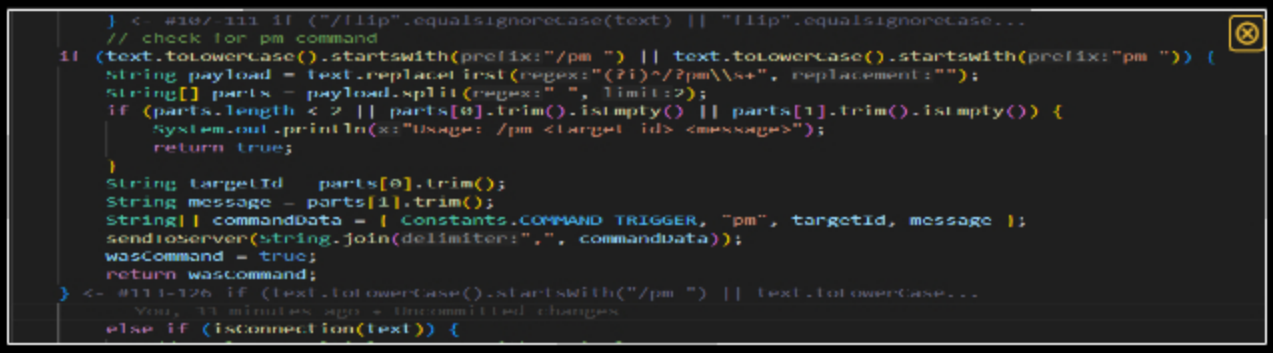
Part 1:

Progress: 100%

Details:
Multiple screenshots are expected

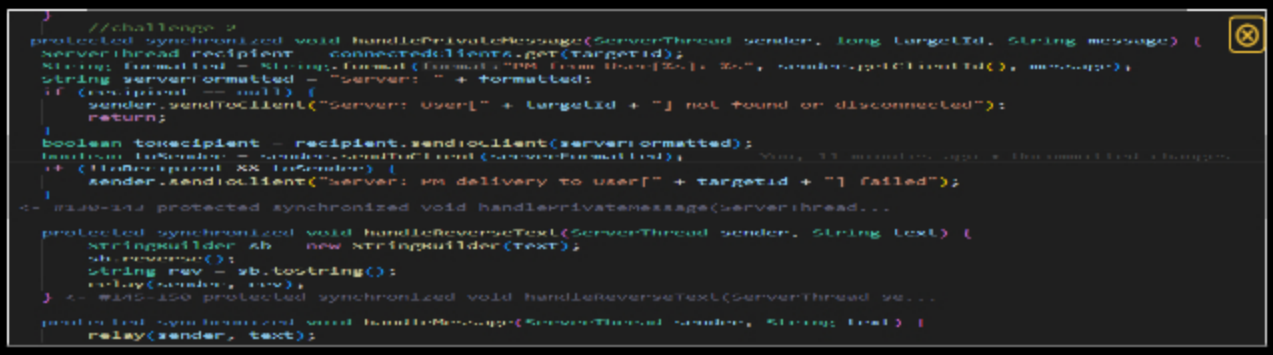
1. Snippet of relevant code showing solution (with ucid/date comment) from `Client`
   - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from `ServerThread`
   - Should only need to edit `processCommand()`

3. Snippet of relevant code showing solution (with ucid/date comment) from `Server`
    - Should only need to create a new method and send the result message to just the sender and receiver
4. Show 3 examples of the command being seen across all terminals (3+ Clients and 1 Server)
    1. This can be captured in one screenshot if you split the terminals side by side
    2. Note: Only the sender and the receiver should see the private message (show variations across different users)



```
        } <- #107 111 if ("/lip".equalsIgnoreCase(text) || "!lip".equalsIgnoreCase...
        // check for pm command
    if (text.toLowerCase().startsWith(prefix:"/pm ") || text.toLowerCase().startsWith(prefix:"pm ")) {
        String payload = text.replaceFirst(regex:"(?i)^/?pm\\s+", replacement:"");
        String[] parts = payload.split(regex:" ", limit:2);
        if (parts.length < 2 || parts[0].trim().isEmpty() || parts[1].trim().isEmpty()) {
            System.out.println(x:"Usage: /pm <target id> <message>");
            return true;
        }
        String targetId   parts[0].trim();
        String message = parts[1].trim();
        String[] commandData = { Constants.COMMAND_TRIGGER, "pm", targetId, message };
        sendToServer(String.join(delimiter:",", commandData));
        wasCommand = true;
        return wasCommand;
    } <- #111-126 if (text.toLowerCase().startsWith("/pm ") || text.toLowerCase...
          You, 33 minutes ago • Uncommitted changes
        else if (isConnection(text)) {
```

client code

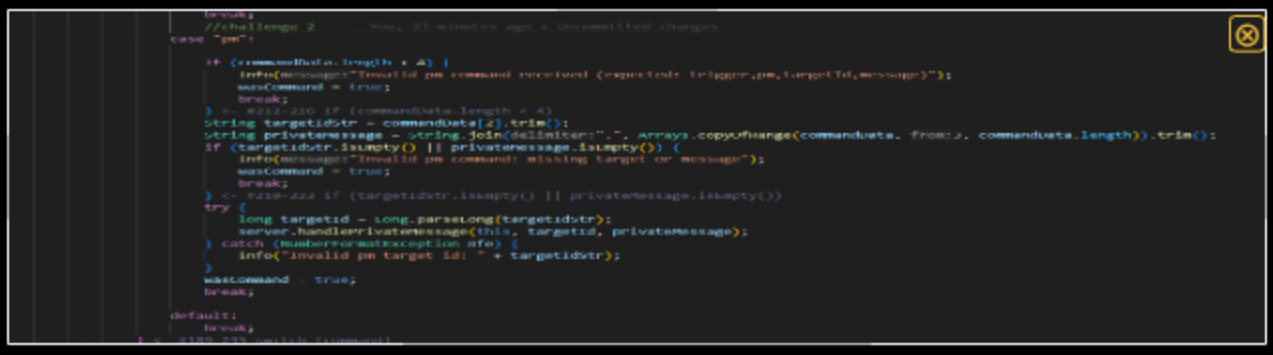


```
    }
        //challenge 2
    protected synchronized void handlePrivateMessage(ServerThread sender, long targetId, String message) {
        ServerThread recipient   connectedClients.get(targetId);
        String formatted = String.format("PM from User[%s]: %s", sender.getClientId(), message);
        String serverFormatted = "Server: " + formatted;
        if (recipient == null) {
            sender.sendToClient("Server: User[" + targetId + "] not found or disconnected");
            return;
        }
        boolean toRecipient = recipient.sendToClient(serverFormatted);
        boolean toSender = sender.sendToClient(serverFormatted);
        if (!toRecipient && !toSender) {
            sender.sendToClient("Server: PM delivery to user[" + targetId + "] failed");
        }
    <- #130-141 protected synchronized void handlePrivateMessage(ServerThread...

    protected synchronized void handleReverseText(ServerThread sender, String text) {
        StringBuilder sb   new StringBuilder(text);
        sb.reverse();
        String rev = sb.toString();
        relay(sender, rev);
    } <- #143-150 protected synchronized void handleReverseText(ServerThread se...

    protected synchronized void handleMessage(ServerThread sender, String text) {
        relay(sender, text);
```

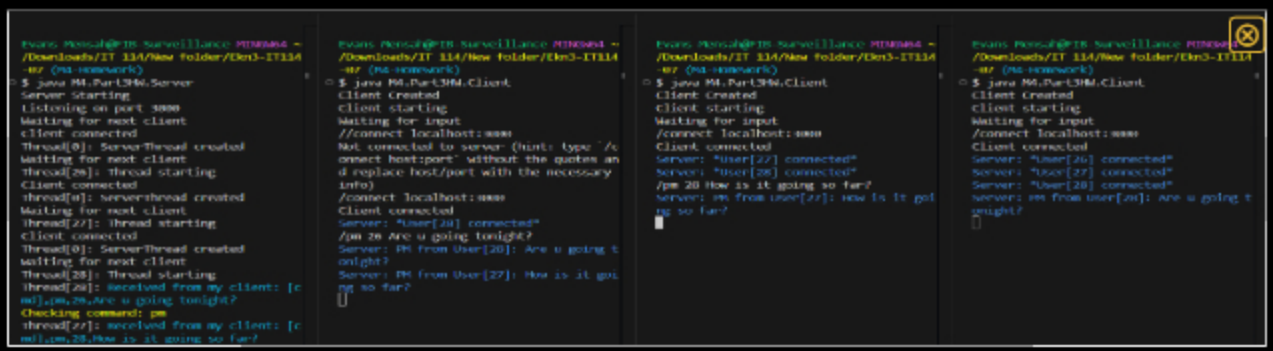
server code



```
                break;
                //challenge 2
            case "pm":
                if (commandData.length < 4) {
                    info(message:"Invalid pm command received (expected: trigger,pm,targetId,message)");
                    wasCommand = true;
                    break;
                } <- #212-216 if (commandData.length < 4)
                String targetIdStr = commandData[2].trim();
                String privateMessage = String.join(delimiter:",", Arrays.copyOfRange(commandData, from:3, commandData.length)).trim();
                if (targetIdStr.isEmpty() || privateMessage.isEmpty()) {
                    info(message:"Invalid pm command: missing target or message");
                    wasCommand = true;
                    break;
                } <- #219-222 if (targetIdStr.isEmpty() || privateMessage.isEmpty())
                try {
                    long targetId = Long.parseLong(targetIdStr);
                    server.handlePrivateMessage(this, targetId, privateMessage);
                } catch (NumberFormatException nfe) {
                    info("Invalid pm target id: " + targetIdStr);
                }
                wasCommand = true;
                break;

            default:
                break;
        } <- #195-233 while (!wasCommand)
```

sever thread code



```
Evans Mensah@PIR Surveillance MINGW64 ~/Downloads/IT 114/New folder/Ebn3-IT114-mr (ma-homework)
$ java M4.Part3HW.Server
Server Starting
Listening on port 3000
Waiting for next client
client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[28]: Thread starting
Client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[27]: Thread starting
Client connected
Thread[0]: ServerThread created
Waiting for next client
Thread[28]: Thread starting
Thread[28]: received from my client: [command],pm,20,Are u going tonight?
Checking command: pm
Thread[27]: received from my client: [command],pm,20,How is it going so far?

Evans Mensah@PIR Surveillance MINGW64 ~/Downloads/IT 114/New folder/Ebn3-IT114-mr (ma-homework)
$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
//connect localhost:3000
Not connected to server (hint: type `/connect host:port` without the quotes and replace host/port with the necessary info)
/connect localhost:3000
Client connected
Server: "User[28] connected"
/pm 20 Are u going tonight?
Server: PM from User[20]: Are u going tonight?
Server: PM from User[27]: How is it going so far?

Evans Mensah@PIR Surveillance MINGW64 ~/Downloads/IT 114/New folder/Ebn3-IT114-mr (ma-homework)
$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
Server: "User[27] connected"
Server: "User[28] connected"
/pm 20 How is it going so far?
Server: PM from User[27]: How is it going so far?

Evans Mensah@PIR Surveillance MINGW64 ~/Downloads/IT 114/New folder/Ebn3-IT114-mr (ma-homework)
$ java M4.Part3HW.Client
Client Created
Client starting
Waiting for input
/connect localhost:3000
Client connected
Server: "User[26] connected"
Server: "User[27] connected"
Server: "User[28] connected"
Server: PM from User[20]: Are u going tonight?
```

Output

Saved: 10/22/2025 12:23:23 AM

Part 2:

Progress: 100%

Details:
Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1
https://github.com/Faculty06/Ekn3-IT1[illegible]M4-Homework/M4/Part3HW/Server.java

URL
https://github.com/Faculty06/Ekn

Saved: 10/22/2025 12:23:23 AM

Part 3:

Progress: 100%

Details:
Briefly explain `how` the code solves the challenges (note: this isn't the same as `what` the code does)

Your Response:

The server thread listens to the pm command, checks if it meets the requirement of the including the target and a message, sends to the server and then broadcasts it to the client

Saved: 10/22/2025 12:23:23 AM

Section #3: (3 pts.) Challenge 3 - Shuffle Message

Progress: 100%

Task #1 (3 pts.) - Implement a Shuffle Message Command

Progress: 100%

Details:

- `Client` must capture the user entry and generate a valid command per the lesson details
  - Command format must be `/shuffle <message>`
- `ServerThread` must receive the data and call the correct method on `Server`
- `Server` must expose a method for the logic and send the result to everyone

- The message must be in the format of `Shuffled from <who>: <shuffled_message>` and be from the Server
- Add code to solve the problem (add/commit as needed)

Part 1:

Progress: 100%

Details:
Multiple screenshots are expected

1. Snippet of relevant code showing solution (with ucid/date comment) from `Client`
   - Should only need to edit `processClientCommands()`
2. Snippet of relevant code showing solution (with ucid/date comment) from `ServerThread`
   - Should only need to edit `processCommand()`
3. Snippet of relevant code showing solution (with ucid/date comment) from `Server`
   - Should only need to create a new method and do similar logic to `relay()`
4. Show 3 examples of the command being seen across all terminals (2+ Clients and 1 Server)
   1. This can be captured in one screenshot if you split the terminals side by side

```
//ckn3 challenge 3
if (text.equalsIgnoreCase(anotherString:"/shuffle") || text.equalsIgnoreCase(anotherString:"shuffle")) {
    System.out.println(x:"Usage: /shuffle <message>");
    return true;
}
if (text.toLowerCase().startsWith(prefix:"/shuffle ") || text.toLowerCase().startsWith(prefix:"shuffle ")) {
    String payload = text.replaceFirst(regex:"(?i)^/?shuffle\\s+", replacement:"");
    String message = payload.trim();
    if (message.isEmpty()) {
        System.out.println(x:"Usage: /shuffle <message>");
        return true;        You, 39 minutes ago • Uncommitted changes
    }
    String[] commandData = { Constants.COMMAND_TRIGGER, "shuffle", message };
    sendToServer(String.join(delimiter:",", commandData));
    wasCommand = true;
    return wasCommand;
} <  #133 144 if (text.toLowerCase().startsWith("/shuffle ") || text.toLowe...
```

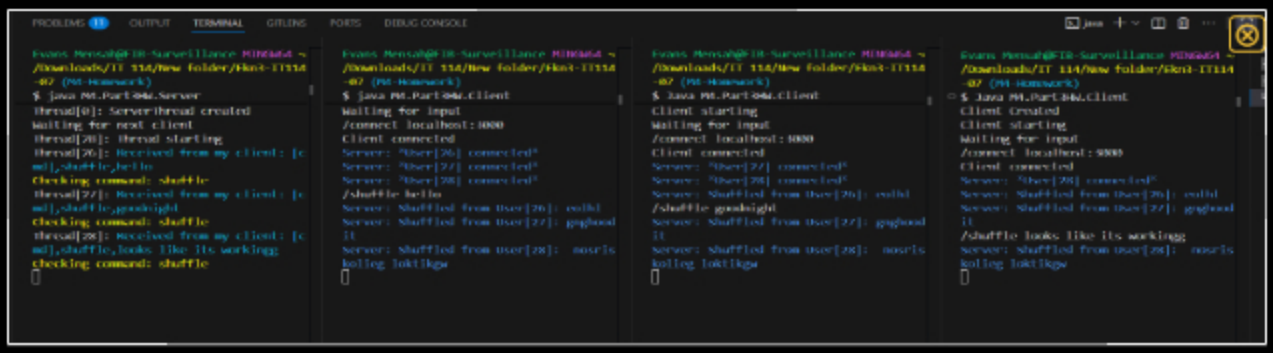
client code

```
//challenge 3
protected synchronized void handleShuffle(ServerThread sender, String message) {
// shuffle characters
char[] chars = message.toCharArray();
java.util.Random rnd = new java.util.Random();
for (int i = chars.length - 1; i > 0; i--) {
    int j = rnd.nextInt(i + 1);
    char tmp = chars[i];
    chars[i] = chars[j];
    chars[j] = tmp;
} <- #160-165 for (int i = chars.length - 1; i > 0; i--)
String shuffled = new String(chars);
String who = sender == null ? "Server" : String.format(format:"User[%s]", sender.getClientId());
String formatted = String.format(format:"Server: shuffled from %s: %s", who, shuffled);
relay(sender:null, String.format(format:"Shuffled from %s: %s", who, shuffled));
<  #156 170 protected synchronized void handleShuffle(ServerThread sender...
```

sever code

```
        //challenge 3
        case "shuffle":
            if (commandData.length < 3) {
                info(message:"Invalid shuffle command received (expected: trigger,shuffle,message)");
                wasCommand = true; // treat as handled so it does not become a broadcast
                break;
            } <- #235-239 if (commandData.length < 3)
            String shuffleMessage = String.join(delimiter:",", Arrays.copyOfRange(commandData, from:2, commandData.length)).trim();
            if (shuffleMessage.isEmpty()) {
                info(message:"Invalid shuffle command: missing message");
                wasCommand = true;
                break;
            } <- #241-245 if (shuffleMessage.isEmpty())
            server.handleShuffle(this, shuffleMessage);
```

sever thread code



Output solution

Saved: 10/22/2025 12:23:13 AM

Part 2:

Progress: 100%

Details:
Direct link to the file in the homework related branch from Github (should end in `.java`)

URL #1
https://github.com/Faculty06/Ekn3-IT1[illegible]M4-Homework/M4/Part3HW/ServerThread.java

URL
https://github.com/Faculty06/Ekn

Saved: 10/22/2025 12:23:13 AM

Part 3:

Progress: 100%

Details:
Briefly explain `how` the code solves the challenges (note: this isn't the same as `what` the code does)

Your Response:

The server thread listens to the shuffle command, shuffles the message, sends it to the server and then broadcasts it to the clients

Saved: 10/22/2025 12:23:13 AM

Section #4: (1 pt.) Misc

Progress: 100%

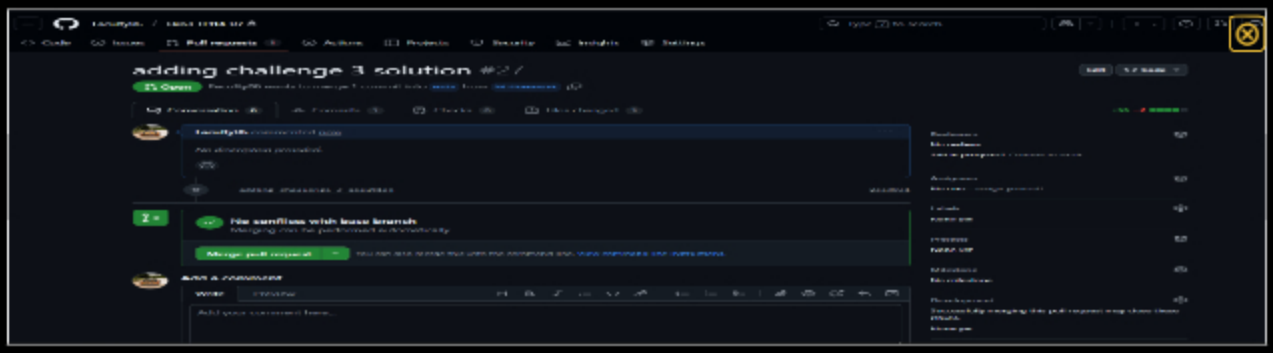
Task #1 (0.33 pts.) - Github Details

Progress: 100%

Part 1:

Progress: 100%

Details:
From the Commits tab of the Pull Request screenshot the commit history Following minimum should be present



pull request

Saved: 10/22/2025 12:26:30 AM

Part 2:

Progress: 100%

Details:
Include the link to the Pull Request (should end in `/pull/#`)

URL #1
https://github.com/Faculty06/Ekn3-IT1 [illegible]

URL
https://github.com/Faculty06/Ekn

Saved: 10/22/2025 12:26:30 AM

Task #2 (0.33 pts.) - WakaTime - Activity

Progress: 100%

Details:

- Visit the WakaTime.com Dashboard
- Click `Projects` and find your repository
- Capture the overall time at the top that includes the repository name
- Capture the individual time at the bottom that includes the file time

- Note: The duration isn't relevant for the grade and the visual graphs aren't necessary



waka time



waka time 2

Saved: 10/22/2025 12:28:51 AM

Task #3 (0.33 pts.) - Reflection

Progress: 100%

Task #1 (0.33 pts.) - What did you learn?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

I figured out how to use the client and server for random stuff like aside these challenges. At first, it looked kinda confusing but not gonna lie, this is fun

Saved: 10/22/2025 12:31:36 AM

Task #2 (0.33 pts.) - What was the easiest part of the

assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The shuffle character. I already knew how to go about it after the stuggling on the first one which isnt that bad now lol

Saved: 10/22/2025 12:33:07 AM

Task #3 (0.33 pts.) - What was the hardest part of the assignment?

Progress: 100%

Details:
Briefly answer the question (at least a few decent sentences)

Your Response:

The first challenge. I guess it was because i had started and needed to warm up

Saved: 10/22/2025 12:34:05 AM